

2.2 项目具体工作

课件智能知识点复习助手的 Vibe Coding 实践记录

本文档仅覆盖 2.2.1 至 2.2.5 小节内容。正文依据当前仓库中的项目代码、开发日志、答辩 PPT、海报、答辩稿和本次完整对话记录整理而成。文中所有需要后补的证据图统一使用占位图框表示，确保在无真实截图的情况下也可以完整编译。

1 2.2.1 项目前期准备

(1) 项目选题与背景

本项目选题来源于课程大作业对“学习辅助工具”的实际需求。传统课程课件通常以 PPT 或 PDF 形式存在，这类资料在教学阶段可以作为讲授载体，但在课后复习阶段往往存在三个显著问题：其一，课件内容长、页面多、重点分散，学生难以快速定位关键知识点；其二，静态课件缺少交互能力，遇到不理解的概念、公式或例题时，学生无法围绕具体上下文持续追问；其三，课后自测通常需要教师额外准备题目与解析，复习链路不完整。

围绕上述问题，本项目希望解决的核心问题是：如何把静态课件升级为一个可复习、可追问、可测试、可导出的学习工作台。最终形成的产品不再只是“看课件”的工具，而是把课件解析、学习笔记、任务规划、研究对话、测试环境和导出结果串联起来的完整复习闭环。

(2) 采用 Vibe Coding 方式开发的理由

本项目最终选择 Vibe Coding 方式开发，主要基于以下考虑。第一，需求本身带有明显的探索性：产品从最初的“知识点提取助手”逐步演化为“知识点复习助手”，中间经历了前端结构重构、对话逻辑重构、数学课件支持增强、模型切换、答辩材料扩展等多次方向调整。第二，本项目同时包含前端界面、后端接口、PDF/PPT 解析、多轮对话、测试题生成等多个模块，人工逐项从零编写会降低迭代速度。第三，项目强调“快速试错 + 连续修正”的开发方式，而 Prompt 驱动的 AI 编程工具非常适合在这个过程中承担骨架搭建、代码初版生成、文档草拟、排版产物生成等工作。

与传统编程方式相比，Vibe Coding 在本项目中的价值主要体现在两点：一是显著缩短了从想法到可运行原型的时间；二是使产品、交互和实现逻辑能够围绕同一条对话链路持续收敛。在本项目中，需求演进、Prompt 设计、代码调整和验证结果是一个高度耦合的连续过程，Vibe Coding 恰好为这种过程提供了合适的方法论基础。

(3) 开发环境说明

本项目开发过程中的核心 AI 编程工具是 Codex，对话式协作是整个开发流程的主线。开发中通过持续输入 Prompt、查看生成结果、进行多轮修正和复检，逐步完成前端、后端、答辩材料和配音产物。技术栈方面，前端采用 React、TypeScript 和 Vite，后端采用 FastAPI 与 SQLite；课件解析层使用 `python-pptx` 和 `PyMuPDF`；文档与答辩材料部分使用 LaTeX、`PptxGenJS`、`Pillow` 等工具链。运行环境为 macOS，本地 Python 环境使用 `conda` 管理，项目同时具备本地前后端运行、PPT 生成、海报生成和答辩稿配音生成能力。

图 2-1 占位：AI 编程工具界面截图 (PLACEHOLDER-AI-TOOL-UI)

建议展示 Codex 对话式开发界面，体现“通过 Prompt 驱动代码生成、修改与验证”的工作方式。

建议来源：与本项目实现相关的 Codex 对话页面截图

图 1: 图 2-1 占位：AI 编程工具界面截图 (PLACEHOLDER-AI-TOOL-UI)

图 2-2 占位：本地项目运行环境或目录结构截图 (PLACEHOLDER-PROJECT-ENV)

建议展示本地项目目录、前后端运行状态，或终端中项目启动界面。

建议来源：项目根目录、运行终端、开发环境界面截图

图 2: 图 2-2 占位：本地项目运行环境或目录结构截图 (PLACEHOLDER-PROJECT-ENV)

(4) 项目目标定义

结合课程要求和项目演化过程，本项目最终确立了以下五条核心目标：

1. 支持上传 PPTX 与文本型 PDF，并能够自动完成文本提取与章节组织；
2. 自动生成结构化学习笔记，内容不仅包含章节讲解，还要包含关键知识点、公式说明和例题步骤；
3. 提供研究对话区，使用户能够围绕“课件 + 学习笔记”继续提问，并支持多轮上下文承接；

4. 提供 Learning Board 与测试环境，支持从摘要、概念、练习到测试的完整复习路径，并能覆盖选择题、填空题和计算题；
5. 支持导出复习记录，形成可追踪的学习产物，便于课程复习、答辩准备和展示。

上述目标的验收标准并不是抽象描述，而是在开发过程中通过可运行界面、端到端流程、真实课件验证、答辩 PPT、配音和海报等多种产物逐步收口。

2 2.2.2 Vibe Coding 开发过程

(1) Prompt 设计与迭代主线

本项目的开发过程可以分为五个关键 Prompt 迭代阶段。

第一阶段：项目方向确定。初始阶段并没有直接锁定“课件智能知识点复习助手”，而是先围绕课程题目范围进行选择。Prompt 的目标是评估多个方向的可行性，最终收敛到“学习的另一种方式”这一题目，并把原本偏“视频转知识点”的思路调整为“课件转知识点/笔记/测试”的更可落地方向。

第二阶段：系统架构规划。在确定方向后，Prompt 不再只问“能不能做”，而是明确约束输入、输出、技术栈和 API 路径，例如把项目固定为 React + FastAPI 双端结构，输入限定为 PPTX/PDF，模型固定走后端统一调用。这一阶段的 Prompt 表达开始明显结构化：目标、接口、数据流、验证条件被同时说明，因此生成结果比早期的概念化建议稳定得多。

第三阶段：前端交互与视觉重构。随着项目实现，用户多次指出现有前端“很丑”“不够简洁”，于是 Prompt 逐步从“做一个页面”演化为“参考 NotebookLM 的 Sources / Chat / Studio 结构”“再转成高端学术工作台”“最后重构成对话驱动的学习研究台”。这里的经验非常典型：当 Prompt 只说“美化一下”，AI 往往给出平均化页面；当 Prompt 明确写出信息架构、视觉取向、交互逻辑、颜色约束和响应式策略时，前端产物质量明显提高。

第四阶段：数学课件支持增强。在用户使用真实课件《罚函数法.pdf》测试后，Prompt 又转向“数学类课件”这一特定场景。新的 Prompt 不再只要求“生成笔记”，而是明确要求支持 PDF 去噪、章节切分、公式抽取、例题整理、计算题生成和导出补全。这个阶段说明：Prompt 越贴近真实问题，AI 越容易生成针对性的实现方案；尤其在数学课件场景中，如果不明确指出“公式、例题、步骤、计算题”，系统就会自动退化为普通摘要器。

第五阶段：对话质量修复与模型切换。用户贴出真实对话记录后，Prompt 又从“如何生成内容”转向“如何修复研究对话区质量”，重点变成问题类型识别、全局/局部检索分流、多轮上下文承接和练习题延续。随后又进一步把模型链路从阿里云百炼 Qwen 切换到 OpenAI compatible Responses API，并最终为答辩 PPT、讲稿、配音和海报继续生成配套材料。这说明 Vibe Coding 在本项目里不是一次性生成代码，而是一种贯穿“需求 - 实现 - 验证 - 交付”的持续协作方式。

图 2-3 占位: Prompt 初始规划记录 (PLACEHOLDER-PROMPT-INIT)

建议放置项目立项阶段关于选题、技术栈和整体架构的 Prompt 截图。

建议来源: 与 Codex 关于项目方向与架构规划的对话截图

图 3: 图 2-3 占位: Prompt 初始规划记录 (PLACEHOLDER-PROMPT-INIT)

图 2-4 占位: 前端重构 Prompt 记录 (PLACEHOLDER-PROMPT-FRONTEND)

建议放置关于 NotebookLM 风格、研究工作台结构、界面重构的 Prompt 截图。

建议来源: 前端重构阶段的关键对话截图

图 4: 图 2-4 占位: 前端重构 Prompt 记录 (PLACEHOLDER-PROMPT-FRONTEND)

图 2-5 占位: 数学课件增强 Prompt 记录 (PLACEHOLDER-PROMPT-MATH)

建议展示围绕《罚函数法.pdf》提出的公式、例题、计算题增强要求。

建议来源: 数学课件支持阶段的关键 Prompt 截图

图 5: 图 2-5 占位: 数学课件增强 Prompt 记录 (PLACEHOLDER-PROMPT-MATH)

图 2-6 占位: 研究对话区质量修复 Prompt 记录 (PLACEHOLDER-PROMPT-CHAT)

建议放置对话退化、上下文承接失败、公式问答乱码等问题修复的 Prompt 截图。

建议来源: 研究对话区质量修复阶段的关键对话截图

图 6: 图 2-6 占位: 研究对话区质量修复 Prompt 记录 (PLACEHOLDER-PROMPT-CHAT)

(2) 功能模块实现说明

从实现层面看，本项目可分为课件处理、问答检索、测试生成和前端交互四个模块。

课件处理模块。后端上传入口负责接收文件、做格式限制、落盘并触发整条处理链路。随后解析层根据文件类型分别处理 PPTX 和 PDF，其中 PDF 解析重点解决目录噪声和标题识别问题。

Listing 1: 文件上传与处理入口：上传课件后触发整条生成链路

```
@app.post("/api/documents/upload", response_model=UploadResponse)
async def upload_document(file: UploadFile = File(...)):
    filename = file.filename or ""
    suffix = Path(filename).suffix.lower()
    if suffix not in {".pptx", ".pdf"}:
        raise HTTPException(status_code=400, detail="仅支持上传 PPTX 或文本型 PDF 。")

    document_id = uuid4().hex[:12]
    saved_path = await _save_upload(document_id, file)
    repository.create_document(
        document_id=document_id,
        title=Path(filename).stem or "未命名课件",
        original_filename=filename,
        file_type=suffix.lstrip("."),
        source_path=str(saved_path.relative_to(settings.root_dir)),
        status="processing",
    )
    detail = process_saved_document(document_id, saved_path)
    return UploadResponse(document=detail)
```

PDF 去噪与章节识别模块。在真实数学课件测试中，系统最初会把整份 PDF 错误聚成一个 section。为修复这一问题，解析器引入了按页行文本提取、重复噪声检测和真实标题识别逻辑。

Listing 2: PDF 去噪与标题识别：解决目录噪声影响章节切分的问题

```
def _parse_pdf(file_path: Path) -> tuple[str, str, list[ParsedFragment]]:
    document = fitz.open(file_path)
    page_lines = [_extract_pdf_lines(page.get_text("text")) for page in document]
    noise_lines = _detect_pdf_noise_lines(page_lines, document.page_count)

    for index, lines in enumerate(page_lines, start=1):
        filtered_lines = _filter_pdf_lines(lines, noise_lines)
        if _should_skip_pdf_page(filtered_lines):
            continue
```

```

text = "\n".join(filtered_lines).strip()
title = _detect_pdf_page_title(filtered_lines)
fragments.append(
    ParsedFragment(
        fragment_index=index,
        source_label=f"第 {index} 页",
        source_type="page",
        title=title,
        text=text,
    )
)

```

研究对话区模块。对话区不是简单把问题丢给模型，而是先做问题类型识别，再根据类型选择上下文构造方式。例如摘要类问题走全局上下文，公式类问题走公式索引，练习题和追问问题则优先承接上一轮例题。

Listing 3: 研究对话区问题类型识别：决定检索与回答策略

```

def _detect_question_type(question: str, recent_chats: list[ChatRecord]) -> str:
    normalized = question.strip().lower()
    if not normalized or re.fullmatch(r"[\d\W_]+", normalized):
        return "clarify"
    if any(token in question for token in ("公式", "数学公式", "推导", "latex", "符号")):
        return "formula"
    if any(token in question for token in ("练习题", "出个题", "出题", "习题")):
        return "exercise"
    if any(token in question for token in ("怎么解决", "怎么做", "如何求解", "怎么解")) and recent_chats:
        return "follow_up"
    if any(token in question for token in ("总结", "概括", "摘要")):
        return "summary"
    return "general"

```

测试环境模块。针对数学课件的需求，测试环境不再局限于概念题，而是支持 calculation 类型题目，并在交卷后提供答案、解析和步骤。

Listing 4: 测试题生成逻辑：支持 calculation 类型

```

class AssessmentQuestion(BaseModel):
    id: str
    type: Literal["choice", "blank", "calculation"]
    prompt: str
    options: list[AssessmentQuestionOption] = Field(default_factory=list)
    answer: str = ""
    display_answer: str = ""

```



```
acceptable_answers: list[str] = Field(default_factory=list)
explanation: str
solution_steps: list[str] = Field(default_factory=list)
```

前端交互模块。前端以三栏工作台为核心，既承担资料展示，也负责 Learning Board 与测试环境联动。当任务完成时，右侧区域自动切换到测试环境。

Listing 5: Learning Board 与测试环境联动：任务完成后自动进入测试区

```
const boardStats = getBoardStats(documentDetail, completedTaskIds);
const isAssessmentUnlocked = boardStats.total > 0 && boardStats.completed >=
  boardStats.total;

useEffect(() => {
  if (!documentDetail || !isAssessmentUnlocked || assessment || assessmentLoading)
  {
    return;
  }
  void loadAssessment(documentDetail.id);
}, [assessment, assessmentLoading, documentDetail, isAssessmentUnlocked]);
```

图 2-7 占位：AI 生成代码对话截图 1 (PLACEHOLDER-AI-CODE-GEN-1)

建议放置 AI 生成或修改后端关键逻辑的对话截图，例如 PDF 去噪或测试题生成。

建议来源：与 Codex 关于关键后端模块实现的对话截图

图 7: 图 2-7 占位：AI 生成代码对话截图 1 (PLACEHOLDER-AI-CODE-GEN-1)

图 2-8 占位：AI 生成代码对话截图 2 (PLACEHOLDER-AI-CODE-GEN-2)

建议放置 AI 生成或修改前端工作台交互、Assessment 联动的对话截图。

建议来源：与 Codex 关于前端交互逻辑实现的对话截图

图 8: 图 2-8 占位：AI 生成代码对话截图 2 (PLACEHOLDER-AI-CODE-GEN-2)

(3) 调试与问题解决记录

在本项目开发过程中，调试并不是零散发生的，而是贯穿于多轮 Prompt 设计与代码重构之中。根据开发日志，可以总结出三类典型问题。

第一类问题是 **数学类 PDF 课件支持不足**。用户以真实课件《罚函数法.pdf》测试后发现，最初系统生成的 Markdown 几乎没有公式和例题，整份 PDF 也只被错误聚成一个章节。对此，开发过程首先通过真实课件检查定位到：PDF 文本层本身包含公式和算法步骤，但标题检测把目录噪声误当成章节标题。随后调整了解析器、章节草稿结构和模型 Prompt，使其增加公式候选、例题候选、公式说明和计算步骤输出，最终让数学课件生成结果更接近真正的复习笔记。

第二类问题是 **研究对话区质量退化**。用户贴出了多轮真实对话，指出系统经常把所有问题都回答成整份课件的泛化总结，甚至像“11”“123”这种低信息输入也会被硬答。为解决这一问题，开发过程对聊天链路进行了系统重构：先识别问题类型，再选择全局或局部检索，再按照不同类型注入对应回答骨架，最后增加低信息输入澄清和上一轮练习题承接逻辑。这一系列修复直接提升了对话区的上下文连续性。

第三类问题是 **模型与配套材料交付链兼容问题**。项目中后期又增加了答辩 PPT、答辩稿、配音和海报等交付物，导致问题从“功能实现”延伸到“完整交付”。例如，最初的 OpenAI compatible 中转只支持 Responses API，无法直接用于 Audio Speech API；最终转而使用阿里云百炼语音合成接口完成逐页音频生成，再拼接为一条总音频。这一过程说明：Vibe Coding 虽然大幅提高了开发速度，但在文档、音频、导出等外围链路上仍然需要工程化验证和人工收口。

3 2.2.3 项目成果展示

(1) 功能完整性验证

截至当前阶段，项目已经形成了一个完整可运行的学习工作台，其成果不仅体现在单个模块是否可用，更体现在这些模块能否串成一个稳定流程。具体来看，成果展示可分为以下几个部分。

课件上传与主工作台。用户上传 PPTX 或 PDF 后，系统能够完成文档接收、落盘、解析和详情展示，主工作台会自动进入资料区、研究对话区和任务区联动状态。

学习笔记生成。系统会把原始课件内容组织为章节概述、详细讲解、关键知识点、核心公式与例题步骤，尤其对数学类课件已经不再只停留在摘要级别。

研究对话区。用户可以围绕课件与笔记继续追问。当前系统支持摘要类、概念类、公式类、练习类和 follow-up 类问题，并能在一定程度上维持多轮上下文。

Learning Board 与测试环境。右侧任务区会展示摘要、概念、练习与复习路径；任务完成后自动进入测试环境，支持选择题、填空题和计算题，并在交卷后显示解析与步骤。

导出、PPT、配音与海报。除了主系统功能外，本项目还额外生成了答辩 PPT、答辩稿、总音频以及项目海报，说明项目已经从“功能实现”扩展到了“完整展示交付”。

图 2-9 占位：主工作台运行效果图 (PLACEHOLDER-UI-MAIN)

建议展示三栏式工作台的整体截图。

建议来源：推荐来源：tmp/current_ui_rewrite_loaded.png

图 9: 图 2-9 占位：主工作台运行效果图 (PLACEHOLDER-UI-MAIN)

图 2-10 占位：学习笔记生成效果图 (PLACEHOLDER-UI-NOTE)

建议展示章节讲解、关键知识点、公式或例题区域。

建议来源：推荐来源：课件上传后生成笔记的界面截图

图 10: 图 2-10 占位：学习笔记生成效果图 (PLACEHOLDER-UI-NOTE)

图 2-11 占位：研究对话区效果图 (PLACEHOLDER-UI-CHAT)

建议展示用户提问与系统围绕课件继续回答的界面。

建议来源：推荐来源：研究对话区截图

图 11: 图 2-11 占位：研究对话区效果图 (PLACEHOLDER-UI-CHAT)

图 2-12 占位：测试环境效果图 (PLACEHOLDER-UI-ASSESSMENT)

建议展示题目、用户作答、交卷后答案与解析。

建议来源：推荐来源：Assessment 区域截图

图 12: 图 2-12 占位：测试环境效果图 (PLACEHOLDER-UI-ASSESSMENT)

(2) 演示视频说明

根据现有系统和答辩材料，可以将演示视频内容结构安排为以下六段：第一段为项目介绍，简要说明项目目标和痛点；第二段演示上传课件，展示主工作台进入可交互状态；第三段查看学习笔记，说明系统如何把静态课件转成结构化笔记；第四段演示研究对话区，展示围绕课件与笔记的多轮提问；第五段展示 Learning Board 与测试环境，说明任务完成后如何进入测验；第六段演示导出结果，说明如何形成完整复习记录。该结构与本项目现有答辩稿和音频成果保持一致，因此在后续录屏时可以直接沿用。

图 2-13 占位：演示视频结构说明图 (PLACEHOLDER-VIDEO-OUTLINE)

建议放置录屏提纲、时间轴草图或录制界面截图。

建议来源：推荐来源：录屏准备界面或答辩稿时间结构截图

图 13: 图 2-13 占位：演示视频结构说明图 (PLACEHOLDER-VIDEO-OUTLINE)

(3) 界面与交互设计说明

本项目界面设计并不是一次完成的，而是在多轮 Prompt 驱动下逐步收敛。最终形成的界面采用三栏工作台结构：左栏突出资料与笔记，中栏突出研究对话区，右栏突出学习任务与测试环境。中栏被设计为整个界面的主舞台，目的是让“围绕课件持续理解”的行为成为用户主路径；左右两栏则承担辅助角色，分别支持资料导航和任务推进。

从用户体验角度看，这种结构有三个优点。第一，上传、查看、提问、测试和导出都发生在同一工作台内，减少了页面跳转带来的认知负担。第二，中栏对话区与右侧任务区之间形成了自然联动，用户可以从任务直接跳回对话区，也可以在对话中继续推进任务。第三，界面风格从早期功能原型逐渐演化为更适合答辩展示的“浅色研究室”风格，使得系统既具有工具属性，也适合作为课程项目成果展示。

4 2.2.4 Vibe Coding 方法论总结

(1) Prompt 工程经验

通过本项目的完整开发过程，可以总结出几条较为稳定的 Prompt 工程经验。

第一，**结构化描述明显优于模糊描述**。在前端重构阶段，如果 Prompt 只写“美化一下页面”，系统通常会生成平均化的布局；而当 Prompt 明确写出信息架构、视觉方向、三栏结构、配色限制和交互逻辑时，生成质量显著提升。第二，**复杂需求拆成阶段目标更稳**

定。无论是系统架构、数学课件支持、对话质量修复，还是答辩材料生成，只要把“本轮目标、输入、输出、验收方式”写清楚，AI 更容易生成可执行结果。第三，**上下文补充对质量影响很大**。数学课件、对话质量、海报和配音这些需求，只有在补充真实案例、限制条件和现有文件路径之后，AI 才能准确生成贴合当前项目的方案。第四，**真实案例回归比简单烟测更有效**。例如《罚函数法.pdf》与用户贴出的真实多轮对话，都是推动系统从“看起来可用”走向“真正可用”的关键。

(2) AI 辅助编程效率分析

从效率角度看，Vibe Coding 相比传统编程方式最大的优势在于“快速搭建 + 快速迭代”。在项目初期，AI 可以非常快地完成目录结构、前后端骨架、接口草稿和基础页面；在中期，AI 特别适合承担多轮前端重构、代码片段扩展、文档起草和答辩材料生成；在后期，AI 又能继续参与海报、PPT、讲稿和配音等附加交付物的制作。这种从“代码实现”扩展到“完整交付”的能力，是传统手工开发难以在短时间内复制的。

但这并不意味着 AI 可以完全代替人工。结合本项目经验，AI 的局限也非常明显：第一，数学公式、数学推导和例题步骤在未加限制时容易生成不严谨内容；第二，截图、PDF、PPT、音频接口等外部交付链路常常受环境和兼容性影响，需要人工验证；第三，真实用户视角的问题，例如“界面很丑”“对话质量很差”，往往必须由人工先提出，再由 AI 协助修改。因此，更准确的理解应该是：Vibe Coding 并不是“让 AI 自动完成一切”，而是让开发者把精力更多放在目标定义、Prompt 设计、质量判断和结果收口上。

(3) 改进空间与反思

从当前项目状态来看，仍然存在若干改进空间。首先，OCR 仍未支持，因此扫描版 PDF 课件无法进入同样的处理链路。其次，数学公式的规范化水平还可以继续增强，当前系统虽然能提取公式并用于问答、测试和笔记，但在排版和符号严谨性上还有继续优化空间。第三，音频与导出链路虽然最终打通，但中间明显受第三方 API 兼容性影响，说明外围交付链仍需更强的工程化封装。第四，文档和截图整理虽然已经开始形成规范流程，但最终课程提交版仍需要人工补图、审校和收口。换句话说，本项目已经证明 Vibe Coding 在开发速度上的优势，但要形成真正高质量的课程成果，仍然需要开发者对结果负责。

5 2.2.5 技术创新点

(1) 创新主题选择

本项目最终选择的主创新点，是**研究对话区的问题类型识别与多轮上下文承接机制**。这一创新并不属于基础上传、解析或笔记生成功能，而是在这些基础能力已经具备之后，进一步提升系统“能不能真正围绕课件持续交互”的能力。

(2) 创新思路

基础方案中的聊天能力，本质上只是“把问题丢给模型”。这样的问题在于：不同类型的问题需要不同的上下文组织方式，系统如果一律使用同一种检索和回答策略，就会退化成整份课件的泛化总结。例如用户想问公式、让系统出一道练习题，或者在上一轮题目基础上继续追问“怎么解决”，如果系统无法识别这些意图，就无法做出真正有用的回答。

因此，本项目的创新思路是：先在对话入口识别问题类型，再根据类型选择不同的上下文构造与回答策略。这样，摘要类问题会走全局上下文，公式类问题会优先走公式索引，练习类问题会围绕例题内容展开，而 follow-up 类问题则会显式承接上一轮内容。

(3) 实现过程

这一创新主要落在后端聊天链路中，包括三个步骤。第一步是问题类型识别，将用户问题区分为 `summary`、`mastery`、`concepts`、`formula`、`exercise`、`follow_up` 和 `clarify` 等类型。第二步是上下文组织：不同问题类型会走全局上下文块、公式上下文块、例题上下文块或上一轮对话焦点。第三步是回答骨架注入：针对不同问题类型，系统为模型注入不同的回答结构，例如公式问答必须输出核心公式及符号含义，练习题必须输出题目、解题思路、详细解答和易错点。

(4) 与基础方案的区别

与基础方案相比，这一创新的关键区别在于：它不再把所有用户问题都当成“普通问答”，而是把不同类型的学习行为显式建模。基础聊天功能的目标只是“能回答”，而这一创新机制的目标是“按照学习场景回答”。前者只能提供信息，后者则更接近一个真正的研究与复习助手。

(5) 效果分析

从现有开发日志与回归结果来看，这一机制至少带来了三方面改进。第一，像“11”“123”这类低信息输入不再被硬答，而是返回澄清提示；第二，公式类问题不再总是退化成整份课件总结，系统可以更稳定地给出可读公式；第三，在“帮我出个练习题”“怎么解决”这类多轮场景中，系统已经能承接上一轮题目继续回答。这说明技术创新点确实建立在基础功能之上，并且对用户体验产生了可感知的改善。

待补截图清单

占位编号	建议内容	推荐来源
PLACEHOLDER-AI-TOOL-UI	AI 编程工具界面截图	Codex 对话界面

占位编号	建议内容	推荐来源
PLACEHOLDER-PROJECT-ENV	项目运行环境或目录结构	本地终端 / Finder / 项目目录
PLACEHOLDER-PROMPT-INIT	项目立项与架构规划 Prompt	初期对话截图
PLACEHOLDER-PROMPT-FRONTEND	前端重构 Prompt	前端多轮重构对话截图
PLACEHOLDER-PROMPT-MATH	数学课件增强 Prompt	围绕罚函数法 PDF 的对话截图
PLACEHOLDER-PROMPT-CHAT	对话质量修复 Prompt	研究对话区修复对话截图
PLACEHOLDER-AI-CODE-GEN-1	后端关键代码生成截图	AI 生成代码对话截图
PLACEHOLDER-AI-CODE-GEN-2	前端关键代码生成截图	AI 生成代码对话截图
PLACEHOLDER-UI-MAIN	主工作台运行效果图	<code>tmp/current_ui_rewrite_loaded.png</code> 对应实机截图
PLACEHOLDER-UI-NOTE	学习笔记生成效果图	章节讲解界面截图
PLACEHOLDER-UI-CHAT	研究对话区效果图	多轮问答截图
PLACEHOLDER-UI-ASSESSMENT	测试环境效果图	Assessment 区截图
PLACEHOLDER-VIDEO-OUTLINE	演示视频结构或录屏提纲	视频录制界面或提纲图
PLACEHOLDER-INNOVATION-ARCH	创新机制结构示意图	聊天链路逻辑图
PLACEHOLDER-INNOVATION-CHAT	创新机制效果截图	公式问答 / 练习追问截图

**图 2-14 占位：创新机制结构示意图
(PLACEHOLDER-INNOVATION-ARCH)**

建议展示“问题类型识别 - 上下文选择 - 回答生成”的结构示意。

建议来源：推荐来源：聊天链路或逻辑流程草图截图

图 14: 图 2-14 占位：创新机制结构示意图 (PLACEHOLDER-INNOVATION-ARCH)

**图 2-15 占位：创新机制问答效果图
(PLACEHOLDER-INNOVATION-CHAT)**

建议展示真实多轮对话中练习题承接或公式问答的界面截图。

建议来源：推荐来源：研究对话区中的多轮交互截图

图 15: 图 2-15 占位：创新机制问答效果图 (PLACEHOLDER-INNOVATION-CHAT)